

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An autonomous Institution affiliated to Anna University)

Degree & Branch	M.Tech Computer Science & Engineering Integrated (5 Years)	Semester V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory	
Academic year	2025–2026 (Odd)	Batch: 2023–2028

Name: I.S.Rajesh

Register No.: 3122237001042

Experiment #2: Loan Amount Prediction using Linear Regression and Support Vector Regression (SVR) Comparison

Aim:

To learn and compare the working of Linear Regression and Support Vector Regression (SVR) with different kernels (Linear, Polynomial, RBF, Sigmoid) on real-world datasets like loan amount prediction.

Libraries used:

numpy, pandas, sklearn, matplotlib, seaborn

Objective:

Apply Linear Regression and Support Vector Regression to predict loan amount sanctioned to users using the provided dataset. Use K-Fold Cross Validation to validate models after effective splitting and compare performance across different SVR kernels.

1 Mathematical Description

Linear Regression

Given data points $(x_1, y_1), \dots, (x_n, y_n)$, Linear Regression fits the model:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

The model parameters β_j are chosen to minimize the Residual Sum of Squares (RSS):

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Ridge Regression

Ridge regression adds L2 regularization (penalty on coefficients):

$$\text{Loss}_{\text{Ridge}} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

Lasso Regression

Lasso regression applies L1 regularization:

$$\text{Loss}_{\text{Lasso}} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

ElasticNet Regression

ElasticNet combines both L1 and L2 penalties:

$$\text{Loss}_{\text{ElasticNet}} = \sum (y_i - \hat{y}_i)^2 + \lambda_1 \sum |\beta_j| + \lambda_2 \sum \beta_j^2$$

Support Vector Regression (SVR)

SVR aims to find a function $f(x)$ that deviates from the target y_i by at most ϵ for each training point, while being as flat as possible. The optimization problem:

$$\min_{w, b, \xi, \xi^*} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*)$$

subject to:

$$\begin{cases} y_i - (w \cdot \phi(x_i) + b) \leq \epsilon + \xi_i \\ (w \cdot \phi(x_i) + b) - y_i \leq \epsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases}$$

where:

- C controls the trade-off between model complexity and ϵ -insensitive loss.
- $\phi(\cdot)$ is the feature mapping defined by the kernel.

SVR Kernels

- **Linear:** $K(x_i, x_j) = x_i^\top x_j$
- **Polynomial:** $K(x_i, x_j) = (\gamma x_i^\top x_j + r)^d$
- **RBF:** $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
- **Sigmoid:** $K(x_i, x_j) = \tanh(\gamma x_i^\top x_j + r)$

K-Fold Cross-Validation

Cross-validation helps estimate a model's generalization to unseen data. In K-Fold CV:

- The dataset is divided into K approximately equal folds.
- The model is trained on $K - 1$ folds and validated on the remaining fold.
- This process repeats K times, each time with a different fold as the validation set.
- Final performance is calculated by averaging the metrics across the K folds.

Mathematically, if E_k is the evaluation metric (e.g., RMSE) from the k^{th} fold, then the overall metric is:

$$E_{\text{avg}} = \frac{1}{K} \sum_{k=1}^K E_k$$

Python Codes for Linear Regression Model

Listing 1: Import libraries

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, root_mean_squared_error
→ , mean_absolute_error, r2_score
```

Listing 2: Numpy

```
# Load Dataset
df = pd.read_csv("train.csv")

# Drop rows with missing target
df = df.dropna(subset=['Loan_Sanction_Amount_(USD)'])

# Fill missing values
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
→ .tolist()
numerical_cols.remove('Loan_Sanction_Amount_(USD)')

categorical_cols = df.select_dtypes(include='object').columns.tolist()

for col in numerical_cols:
    df[col] = df[col].fillna(df[col].median())

# Simultaneously encode categorical variables

for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```

# Target-Guided Ordinal Encoding for Categorical Columns
for col in categorical_cols:
    ordering = df.groupby(col)["Loan_Sanction_Amount_(USD)"].mean().
        ↪ sort_values().index
    ordinal_map = {key: idx for idx, key in enumerate(ordering)}
    df[col] = df[col].map(ordinal_map)

# EDA

# 1. Distribution of Loan Amount
plt.figure(figsize=(6, 4))
sns.histplot(df['Loan_Sanction_Amount_(USD)'], kde=True, color='blue')
plt.title("Loan_Amount_Sanctioned_-_Distribution")
plt.show()

# 2. Boxplots for numerical features
for col in ['Income_(USD)', 'Credit_Score', 'Loan_Amount_Request_(USD)',
    ↪ , 'Property_Price']:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot_of_{col}")
    plt.show()

# 3. Correlation Heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation_Matrix")
plt.show()

# 4. Scatter plots
plt.figure(figsize=(6, 4))
sns.scatterplot(x=df['Income_(USD)'], y=df['Loan_Sanction_Amount_(USD)'],
    ↪ ])
plt.title("Income_vs_Loan_Sanction_Amount")
plt.show()

plt.figure(figsize=(6, 4))
sns.scatterplot(x=df['Credit_Score'], y=df['Loan_Sanction_Amount_(USD)'],
    ↪ ])
plt.title("Credit_Score_vs_Loan_Sanction_Amount")
plt.show()

# Feature Engineering
df['DebtToIncomeRatio'] = df['Current_Loan_Expenses_(USD)'] / (df['
    ↪ Income_(USD)'] + 1)
df['LoanToPropertyRatio'] = df['Loan_Amount_Request_(USD)'] / (df['
    ↪ Property_Price'] + 1)
df['TotalFinancialBurden'] = df['Current_Loan_Expenses_(USD)'] + df['
    ↪ Loan_Amount_Request_(USD)']
df.drop(['Customer_ID', 'Name', 'Gender', 'Property_ID'], axis=1,
    ↪ inplace=True)

df.to_csv('updated_train.csv', index=False)

# Dataset size (after prepreprocessing)
print(df.shape)

```

```

# Define features and target
X = df.drop('Loan_Sanction_Amount_(USD)', axis=1)
y = df['Loan_Sanction_Amount_(USD)']

# Normalize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Splitting: Train (60%), Validation (20%), Test (20%)
X_train, X_temp, y_train, y_temp = train_test_split(X_scaled, y,
    ↪ test_size=0.4, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
    ↪ test_size=0.5, random_state=42)

# Train Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predictions
y_val_pred = model.predict(X_val)
y_test_pred = model.predict(X_test)

# Evaluating Model on Test and Validation Sets (Without Performance
    ↪ Metrics)

def plot_actual_vs_predicted(y_true, y_pred, title):
    plt.figure(figsize=(6, 4))
    plt.scatter(y_true, y_pred, alpha=0.5, edgecolor='k')
    plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()]
    ↪ ], 'r--')
    plt.xlabel("Actual_Loan_Amount")
    plt.ylabel("Predicted_Loan_Amount")
    plt.title(f"Actual_vs_Predicted_{title}")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def plot_residuals(y_true, y_pred, title):
    residuals = y_true - y_pred
    plt.figure(figsize=(6, 4))
    plt.scatter(y_pred, residuals, alpha=0.5, edgecolor='k')
    plt.axhline(y=0, color='red', linestyle='--')
    plt.xlabel("Predicted_Loan_Amount")
    plt.ylabel("Residuals")
    plt.title(f"Residual_Plot_{title}")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def plot_residual_distribution(y_true, y_pred, title):
    residuals = y_true - y_pred
    plt.figure(figsize=(6, 4))
    sns.histplot(residuals, kde=True, color='skyblue')
    plt.title(f"Residual_Distribution_{title}")
    plt.xlabel("Residuals")
    plt.ylabel("Frequency")
    plt.grid(True)
    plt.tight_layout()

```

```

plt.show()

# For Validation Set
plot_actual_vs_predicted(y_val, y_val_pred, "Validation_Set")
plot_residuals(y_val, y_val_pred, "Validation_Set")
plot_residual_distribution(y_val, y_val_pred, "Validation_Set")

# For Test Set
plot_actual_vs_predicted(y_test, y_test_pred, "Test_Set")
plot_residuals(y_test, y_test_pred, "Test_Set")
plot_residual_distribution(y_test, y_test_pred, "Test_Set")

# Performance Metrics
mse = mean_squared_error(y_test, y_test_pred)
rmse = root_mean_squared_error(y_test, y_test_pred)
mae = mean_absolute_error(y_test, y_test_pred)
r2 = r2_score(y_test, y_test_pred)
n, p = X_test.shape
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

print("Mean_Squared_Error:", mse)
print("Root_Mean_Squared_Error:", rmse)
print("Mean_Absolute_Error:", mae)
print("R2_Score:", r2)
print("Adjusted_R2_Score:", adjusted_r2)

# Visualization of Results

# Plot: Actual vs Predicted
plt.figure(figsize=(6, 4))
plt.scatter(y_test, y_test_pred, alpha=0.5)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual_vs_Predicted_Loan_Amount")
plt.show()

# Plot: Residuals
residuals = y_test - y_test_pred
plt.figure(figsize=(6, 4))
sns.histplot(residuals, kde=True, color='orange')
plt.title("Residuals_Distribution")
plt.show()

# Plot: Feature Coefficients
coefficients = model.coef_
feature_names = X.columns
plt.figure(figsize=(10, 6))
sns.barplot(x=coefficients, y=feature_names)
plt.title("Linear_Regression_Coefficients")
plt.xlabel("Coefficient_Value")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()

# Performance Metrics for Validation to compare with Test set and
→ conclude on Overfitting or Underfitting
mse = mean_squared_error(y_val, y_val_pred)

```

```

rmse = root_mean_squared_error(y_val, y_val_pred)
mae = mean_absolute_error(y_val, y_val_pred)
r2 = r2_score(y_val, y_val_pred)
n, p = X_test.shape
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

print("Mean_Squared_Error:", mse)
print("Root_Mean_Squared_Error:", rmse)
print("Mean_Absolute_Error:", mae)
print("R2_Score:", r2)
print("Adjusted_R2_Score:", adjusted_r2)

```

Output for Linear Regression Model

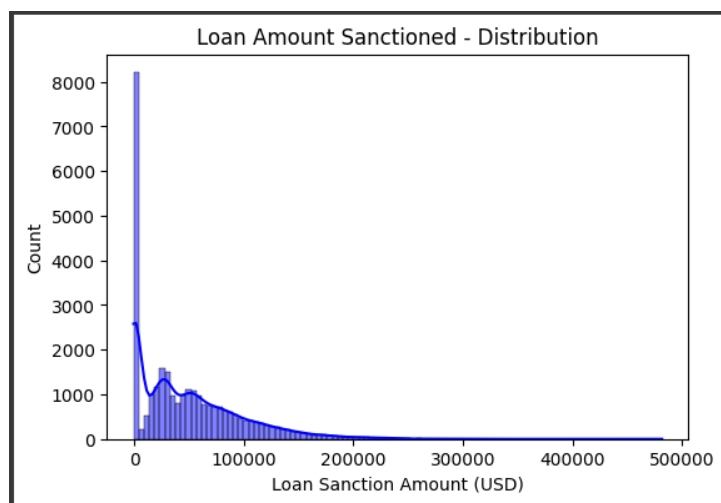


Figure 1: Exploratory Data Analysis 1

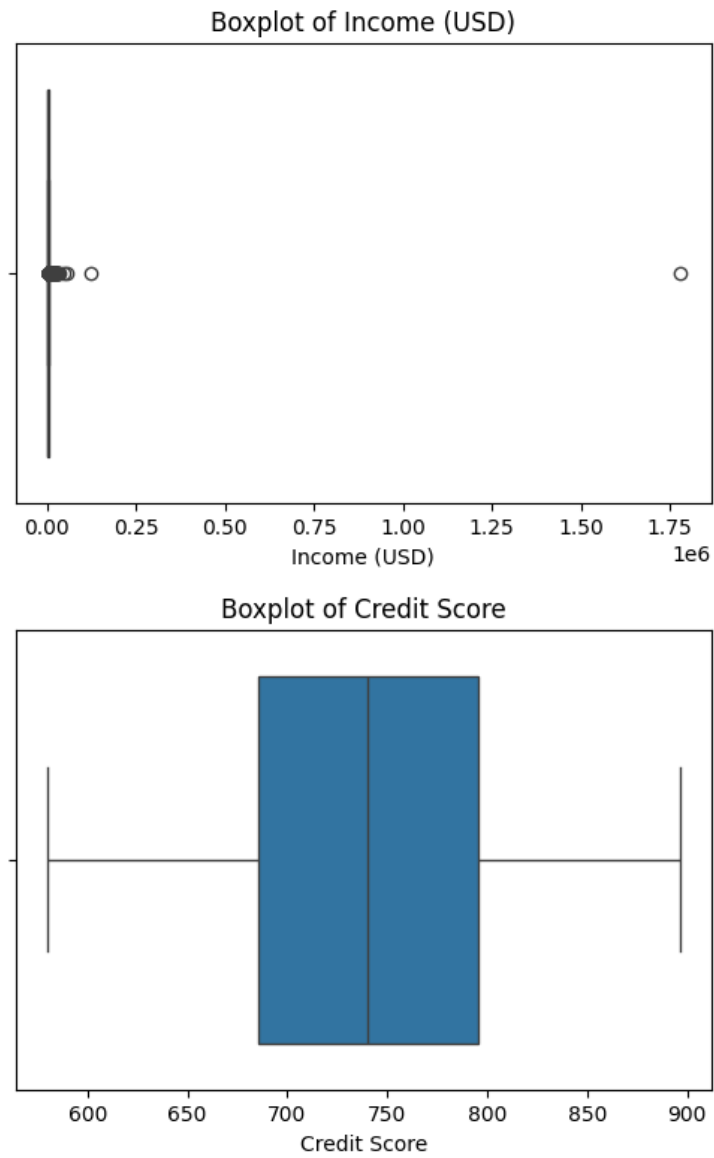


Figure 2: Exploratory Data Analysis 2

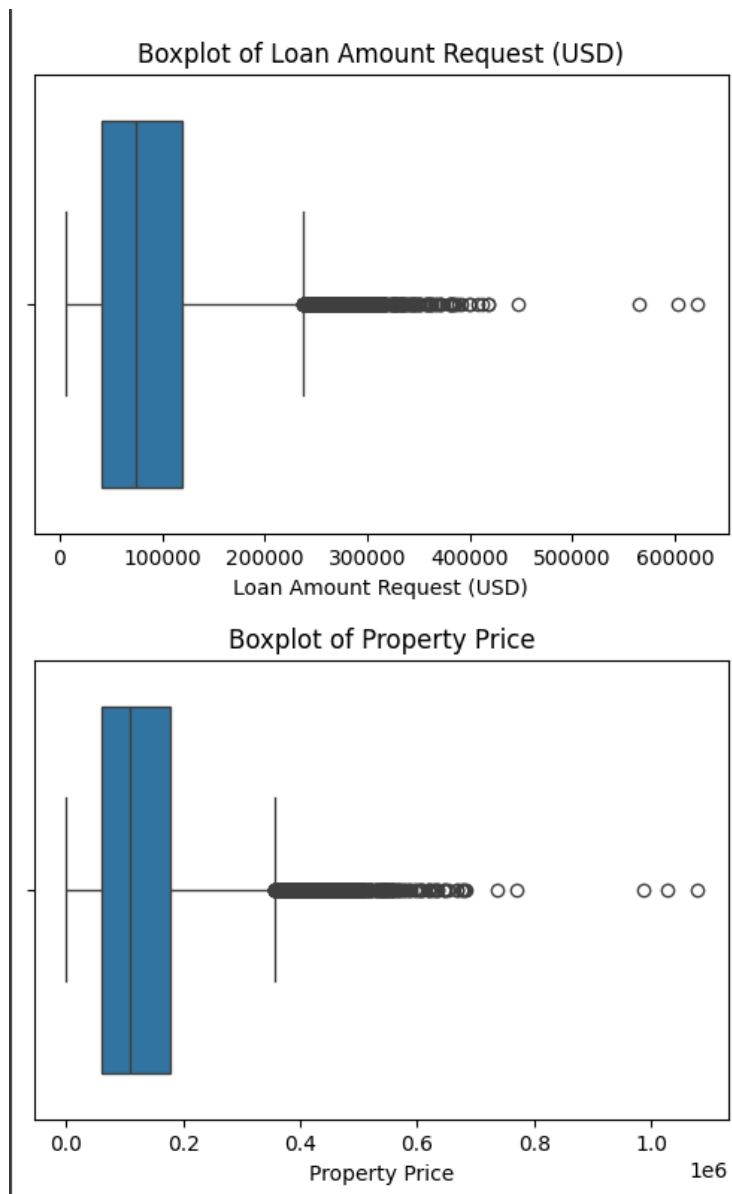


Figure 3: Exploratory Data Analysis 3

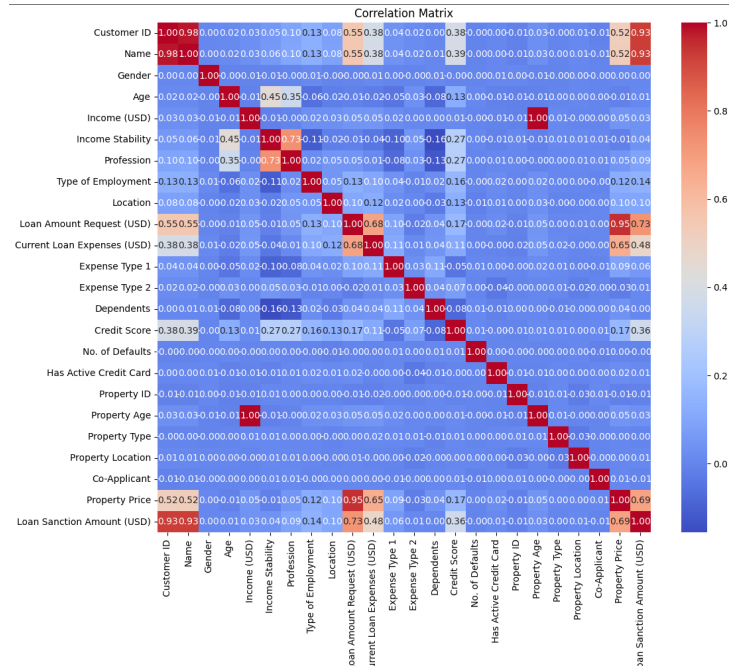


Figure 4: Exploratory Data Analysis 4

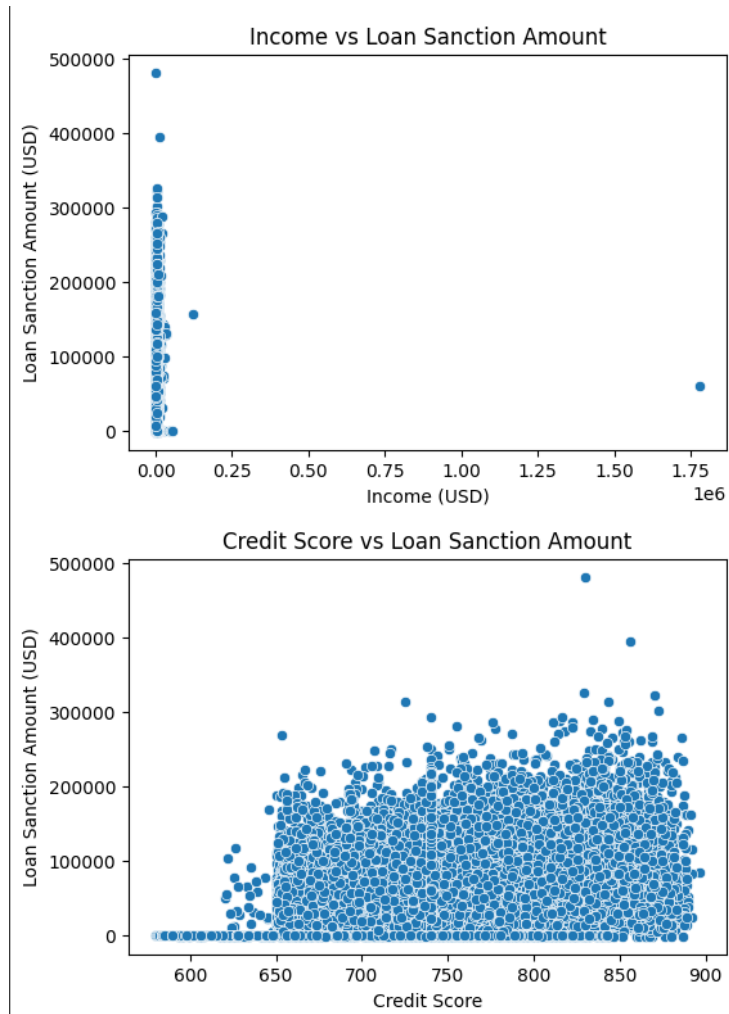


Figure 5: Exploratory Data Analysis 5

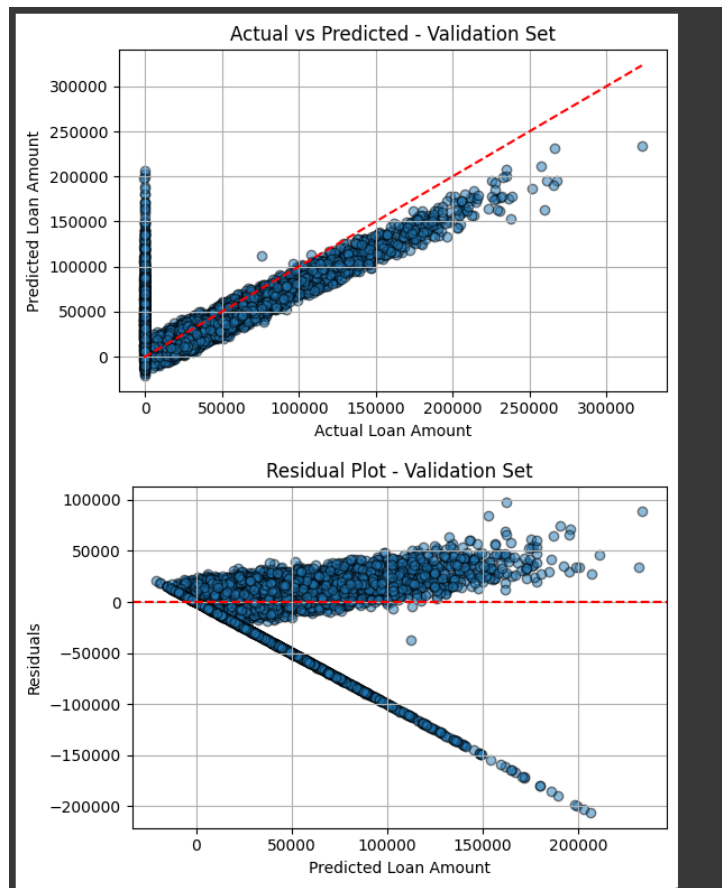


Figure 6: Evaluation without Metrics (Test vs Validation) 1

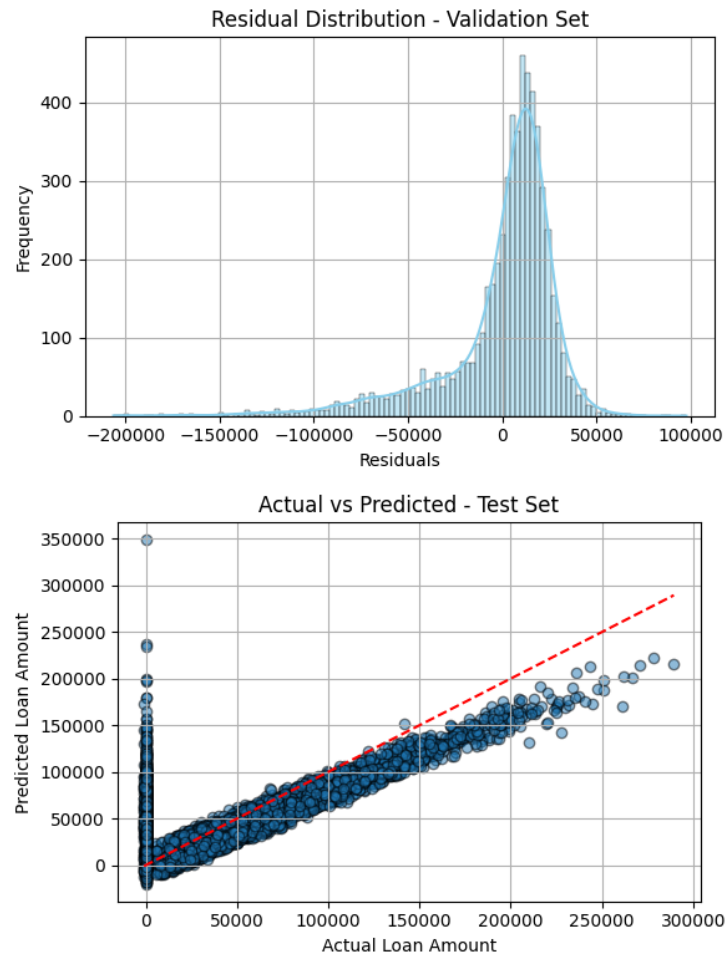


Figure 7: Evaluation without Metrics (Test vs Validation) 2

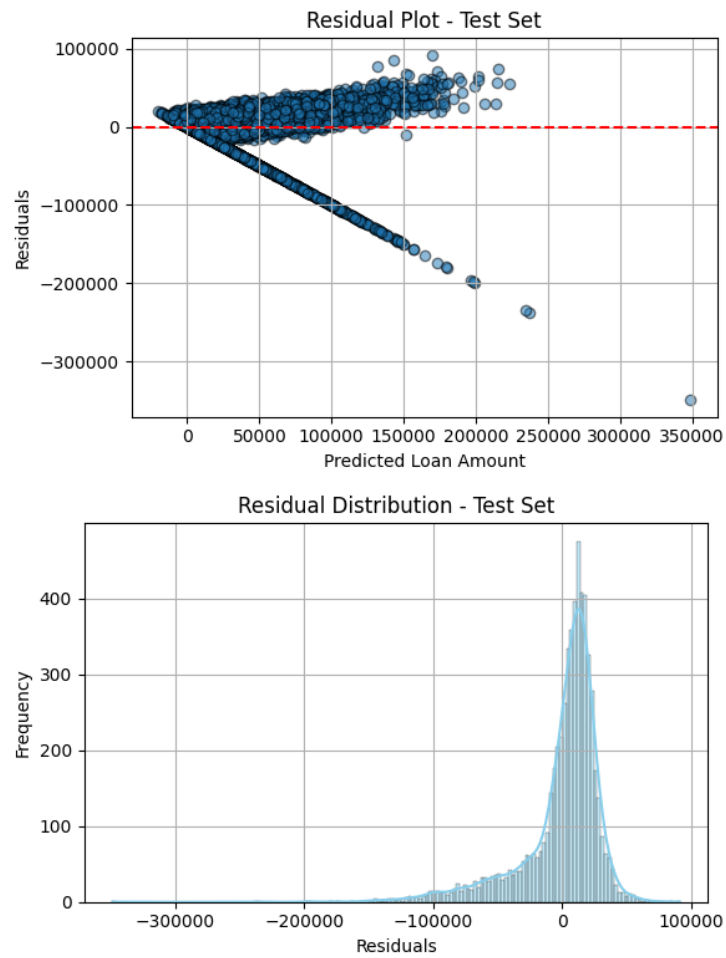


Figure 8: Evaluation without Metrics (Test vs Validation) 3

```

Mean Squared Error: 1017408194.6607875
Root Mean Squared Error: 31896.836750072685
Mean Absolute Error: 21714.30955318851
R2 Score: 0.5499689011897946
Adjusted R2 Score: 0.5482933750138216

```

Figure 9: Test Metrics

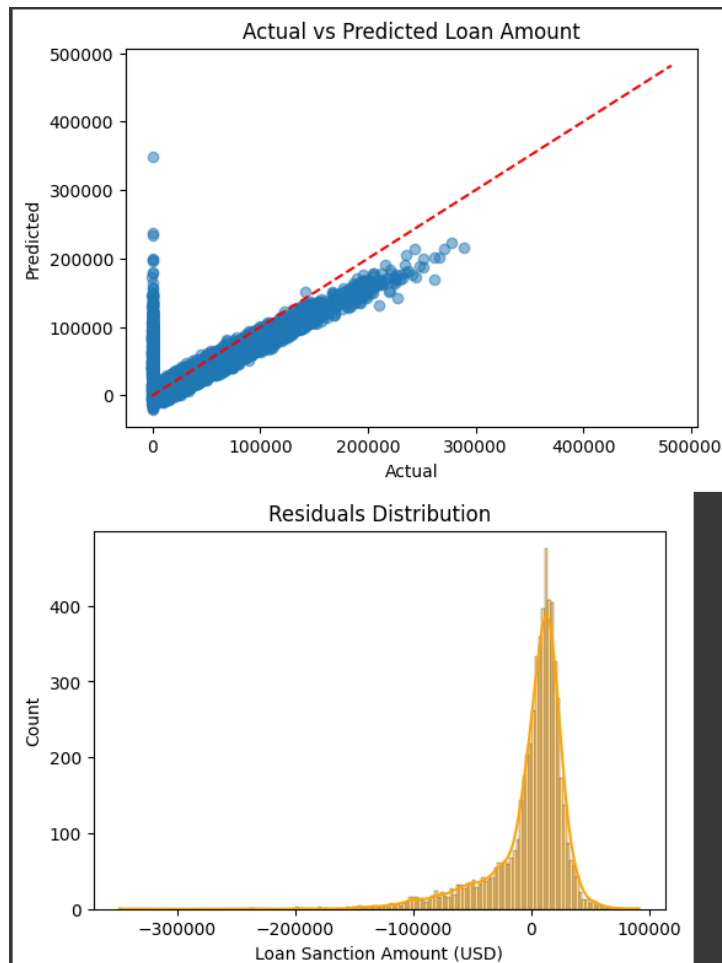


Figure 10: Result Visualization

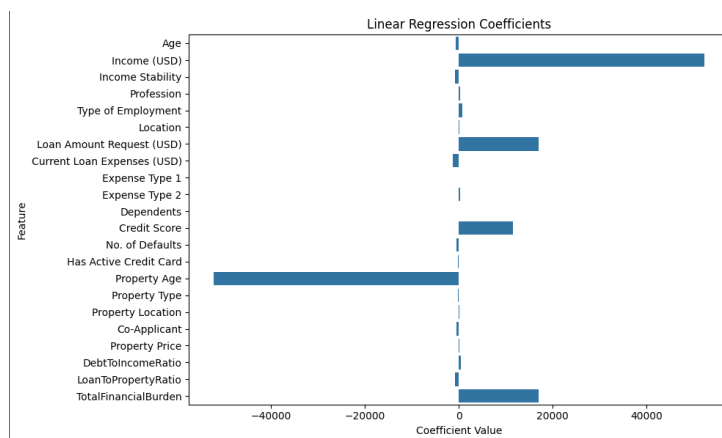


Figure 11: Feature Coefficient Comparison

```
Mean Squared Error: 969797426.9689919
Root Mean Squared Error: 31141.570720967044
Mean Absolute Error: 21308.716483688484
R2 Score: 0.5742636441511726
Adjusted R2 Score: 0.5726785705636495
```

Figure 12: Validation Metrics

Python Codes for K-Fold Cross Validation

Listing 3: Import libraries

```
# Import Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import KFold
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, root_mean_squared_error
→ , mean_absolute_error, r2_score
```

Listing 4: K-Fold Cross Validation

```
# Load the updated dataset
df = pd.read_csv("updated_train.csv")

# Define the target column
target_column = 'Loan_Sanction_Amount_(USD)'

# Separate features and target
X = df.drop(columns=[target_column])
y = df[target_column]

# Standardize the feature values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Initialize 5-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Store evaluation metrics
results = []

# Run cross-validation
for fold, (train_index, val_index) in enumerate(kf.split(X_scaled), 1):
    X_train, X_val = X_scaled[train_index], X_scaled[val_index]
    y_train, y_val = y.iloc[train_index], y.iloc[val_index]

    # Train the model
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Predict on validation set
    y_pred = model.predict(X_val)

    # Calculate metrics
```

```

mse = mean_squared_error(y_val, y_pred)
rmse = root_mean_squared_error(y_val, y_pred)
mae = mean_absolute_error(y_val, y_pred)
r2 = r2_score(y_val, y_pred)
adj_r2 = 1 - (1 - r2) * (len(y_val) - 1) / (len(y_val) - X_val.
    ↪ shape[1] - 1)

results.append({
    "Fold": fold,
    "MAE": mae,
    "MSE": mse,
    "RMSE": rmse,
    "R2": r2,
    "Adjusted_R2": adj_r2
})

cv_results_df = pd.DataFrame(results)

average_row = {
    "Fold": "Average",
    "MAE": cv_results_df["MAE"].mean(),
    "MSE": cv_results_df["MSE"].mean(),
    "RMSE": cv_results_df["RMSE"].mean(),
    "R2": cv_results_df["R2"].mean(),
    "Adjusted_R2": cv_results_df["Adjusted_R2"].mean()
}

cv_results_df = pd.concat([cv_results_df, pd.DataFrame([average_row])],
    ↪ ignore_index=True)

# Display results

# Display results
print(cv_results_df)

```

Output for K-Fold Cross Validation

	Fold	MAE	MSE	RMSE	R2	Adjusted R2
0	1	21539.365959	1.015307e+09	31863.886748	0.552684	0.551018
1	2	21627.319510	9.686148e+08	31122.576996	0.573056	0.571466
2	3	21507.681453	1.181087e+09	34366.951680	0.488430	0.486525
3	4	21511.178652	9.179253e+08	30297.281442	0.620162	0.618748
4	5	21709.723065	9.933505e+08	31517.464130	0.579387	0.577821
5	Average	21579.053728	1.015257e+09	31833.632199	0.562744	0.561116

Figure 13: K-Fold Cross Validation Metrics

Python Codes for Linear Regression Variations

Listing 5: Import libraries

```

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```



```

import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error, root_mean_squared_error
    ↪ , mean_absolute_error, r2_score

```

Listing 6: Variations codes

```

import pandas as pd
import numpy as np
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    ↪ r2_score

# Load dataset
df = pd.read_csv("updated_train.csv")

# Target and Features
target_column = 'Loan_Sanction_Amount_(USD)'
X = df.drop(columns=[target_column])
y = df[target_column]

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
    ↪ test_size=0.2, random_state=42)

# Define models with hyperparameter variations
models = {
    "Ridge_(alpha=0.1)": Ridge(alpha=0.1),
    "Ridge_(alpha=10)": Ridge(alpha=10),
    "Lasso_(alpha=0.1)": Lasso(alpha=0.1, max_iter=10000),
    "Lasso_(alpha=10)": Lasso(alpha=10, max_iter=10000),
    "ElasticNet_(alpha=0.1, l1=0.5)": ElasticNet(alpha=0.1, l1_ratio
    ↪ =0.5, max_iter=10000),
    "ElasticNet_(alpha=10, l1=0.5)": ElasticNet(alpha=10, l1_ratio=0.5,
    ↪ max_iter=10000)
}

# Evaluate and store results
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    adj_r2 = 1 - (1 - r2) * (len(y_test) - 1) / (len(y_test) - X_test.
    ↪ shape[1] - 1)

```

```

results.append({
    "Model": name,
    "MAE": mae,
    "MSE": mse,
    "RMSE": rmse,
    "R2": r2,
    "Adjusted_R2": adj_r2
})

# Display results
results_df = pd.DataFrame(results).sort_values(by="R2", ascending=False
→ )
print(results_df)

```

Output for Linear Regression Variations

	Model	MAE	MSE	RMSE	\
4	ElasticNet ($\alpha=0.1$, $\text{li}=0.5$)	21586.598637	1.011990e+09	31811.791232	
1	Ridge ($\alpha=10$)	21526.591470	1.015082e+09	31860.345738	
3	Lasso ($\alpha=10$)	21523.251271	1.015110e+09	31860.790166	
2	Lasso ($\alpha=0.1$)	21536.573671	1.015232e+09	31862.712494	
0	Ridge ($\alpha=0.1$)	21538.784781	1.015289e+09	31863.603208	
5	ElasticNet ($\alpha=10$, $\text{li}=0.5$)	29304.607142	1.448780e+09	38062.841465	

	R2	Adjusted R2
4	0.554145	0.552485
1	0.552783	0.551118
3	0.552771	0.551105
2	0.552717	0.551051
0	0.552692	0.551026
5	0.361708	0.359331

Figure 14: Variations Metrics

Python Codes for Support Vector Regression (SVR)

Listing 7: SVR Model Implementation

```

# ----- Linear Kernel -----
param_grid_l = {'C': [0.1, 1.0, 10]}
grid_l = GridSearchCV(SVR(kernel='linear'), param_grid_l, cv=5)
grid_l.fit(X_train, y_train)

model_l = grid_l.best_estimator_
y_val_pred_l = model_l.predict(X_val)
y_test_pred_l = model_l.predict(X_test)

# ----- Polynomial Kernel -----
param_grid_p = {
    'C': [0.1, 1.0, 10],
    'degree': [2, 3, 4],
    'gamma': ['scale', 'auto']
}
grid_p = GridSearchCV(SVR(kernel='poly'), param_grid_p, cv=5)
grid_p.fit(X_train, y_train)

model_p = grid_p.best_estimator_

```

```

y_val_pred_p = model_p.predict(X_val)
y_test_pred_p = model_p.predict(X_test)

# ----- RBF Kernel -----
param_grid_r = {
    'C': [0.1, 1.0, 10],
    'gamma': ['scale', 'auto']
}
grid_r = GridSearchCV(SVR(kernel='rbf'), param_grid_r, cv=5)
grid_r.fit(X_train, y_train)

model_r = grid_r.best_estimator_
y_val_pred_r = model_r.predict(X_val)
y_test_pred_r = model_r.predict(X_test)

# ----- Sigmoid Kernel -----
param_grid_s = {
    'C': [0.1, 1.0, 10],
    'gamma': ['scale', 'auto']
}
grid_s = GridSearchCV(SVR(kernel='sigmoid'), param_grid_s, cv=5)
grid_s.fit(X_train, y_train)

model_s = grid_s.best_estimator_
y_val_pred_s = model_s.predict(X_val)
y_test_pred_s = model_s.predict(X_test)

# Evaluating Model using Performance Metrics

def evaluate_model(y_true, y_pred, is_classification, X, model,
    ↪ dataset_name):
    print(f"\nEvaluation_{dataset_name}")
    n, p = X.shape
    r2 = r2_score(y_true, y_pred)
    adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
    print("Mean Squared Error:", mean_squared_error(y_true, y_pred))
    print("Root Mean Squared Error:", root_mean_squared_error(y_true,
    ↪ y_pred))
    print("Mean Absolute Error:", mean_absolute_error(y_true, y_pred))
    print("R2 Score:", r2)
    print("Adjusted R2 Score:", adjusted_r2)

evaluate_model(y_val, y_val_pred_l, True, X_val, model_l, "Validation_
    ↪ Set_Linear_SVM")
evaluate_model(y_test, y_test_pred_l, True, X_test, model_l, "Test_Set_
    ↪ _Linear_SVM")

evaluate_model(y_val, y_val_pred_p, True, X_val, model_p, "Validation_
    ↪ Set_Polynomial_SVM")
evaluate_model(y_test, y_test_pred_p, True, X_test, model_p, "Test_Set_
    ↪ _Polynomial_SVM")

evaluate_model(y_val, y_val_pred_r, True, X_val, model_r, "Validation_
    ↪ Set_RBF_SVM")
evaluate_model(y_test, y_test_pred_r, True, X_test, model_r, "Test_Set_
    ↪ _RBF_SVM")

```

```

evaluate_model(y_val, y_val_pred_s, True, X_val, model_s, "Validation_
    ↳ Set_␣Sigmoid_␣SVM")
evaluate_model(y_test, y_test_pred_s, True, X_test, model_s, "Test_␣Set_
    ↳ _␣Sigmoid_␣SVM")

# Evaluating Model on Test and Validation Sets (Without Performance
    ↳ Metrics)

def plot_actual_vs_predicted(y_true, y_pred, title):
    plt.figure(figsize=(6, 4))
    plt.scatter(y_true, y_pred, alpha=0.5, edgecolor='k')
    plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()]
        ↳ ], 'r--')
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.title(f"Actual_␣vs_␣Predicted_␣-␣{title}")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def plot_residuals(y_true, y_pred, title):
    residuals = y_true - y_pred
    plt.figure(figsize=(6, 4))
    plt.scatter(y_pred, residuals, alpha=0.5, edgecolor='k')
    plt.axhline(y=0, color='red', linestyle='--')
    plt.xlabel("Predicted")
    plt.ylabel("Residuals")
    plt.title(f"Residual_␣Plot_␣-␣{title}")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def plot_residual_distribution(y_true, y_pred, title):
    residuals = y_true - y_pred
    plt.figure(figsize=(6, 4))
    sns.histplot(residuals, kde=True, color='skyblue')
    plt.title(f"Residual_␣Distribution_␣-␣{title}")
    plt.xlabel("Residuals")
    plt.ylabel("Frequency")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# --- Linear SVR ---
plot_actual_vs_predicted(y_val, y_val_pred_l, "Linear_␣SVR_␣-␣Validation_
    ↳ Set")
plot_residuals(y_val, y_val_pred_l, "Linear_␣SVR_␣-␣Validation_␣Set")
plot_residual_distribution(y_val, y_val_pred_l, "Linear_␣SVR_␣-␣
    ↳ Validation_␣Set")

plot_actual_vs_predicted(y_test, y_test_pred_l, "Linear_␣SVR_␣-␣Test_␣Set"
    ↳ )
plot_residuals(y_test, y_test_pred_l, "Linear_␣SVR_␣-␣Test_␣Set")
plot_residual_distribution(y_test, y_test_pred_l, "Linear_␣SVR_␣-␣Test_
    ↳ Set")

# --- Polynomial SVR ---

```

```

plot_actual_vs_predicted(y_val, y_val_pred_p, "Polynomial_SVR-Validation_Set")
plot_residuals(y_val, y_val_pred_p, "Polynomial_SVR-Validation_Set")
plot_residual_distribution(y_val, y_val_pred_p, "Polynomial_SVR-Validation_Set")

plot_actual_vs_predicted(y_test, y_test_pred_p, "Polynomial_SVR-Test_Set")
plot_residuals(y_test, y_test_pred_p, "Polynomial_SVR-Test_Set")
plot_residual_distribution(y_test, y_test_pred_p, "Polynomial_SVR-Test_Set")

# --- RBF SVR ---
plot_actual_vs_predicted(y_val, y_val_pred_r, "RBF_SVR-Validation_Set")
plot_residuals(y_val, y_val_pred_r, "RBF_SVR-Validation_Set")
plot_residual_distribution(y_val, y_val_pred_r, "RBF_SVR-Validation_Set")

plot_actual_vs_predicted(y_test, y_test_pred_r, "RBF_SVR-Test_Set")
plot_residuals(y_test, y_test_pred_r, "RBF_SVR-Test_Set")
plot_residual_distribution(y_test, y_test_pred_r, "RBF_SVR-Test_Set")

# --- Sigmoid SVR ---
plot_actual_vs_predicted(y_val, y_val_pred_s, "Sigmoid_SVR-Validation_Set")
plot_residuals(y_val, y_val_pred_s, "Sigmoid_SVR-Validation_Set")
plot_residual_distribution(y_val, y_val_pred_s, "Sigmoid_SVR-Validation_Set")

plot_actual_vs_predicted(y_test, y_test_pred_s, "Sigmoid_SVR-Test_Set")
plot_residuals(y_test, y_test_pred_s, "Sigmoid_SVR-Test_Set")
plot_residual_distribution(y_test, y_test_pred_s, "Sigmoid_SVR-Test_Set")

```

Output for Support Vector Regression (SVR)

```
Evaluation - Validation Set - Linear SVM
Mean Squared Error: 1201017152.1126287
Root Mean Squared Error: 34655.69436777495
Mean Absolute Error: 19743.97531673635
R2 Score: 0.5151275043042227
Adjusted R2 Score: 0.5073012424514662

Evaluation - Test Set - Linear SVM
Mean Squared Error: 1204795982.1265874
Root Mean Squared Error: 34710.17116245017
Mean Absolute Error: 18811.11986166312
R2 Score: 0.4598233319665932
Adjusted R2 Score: 0.45110441289342007

Evaluation - Validation Set - Polynomial SVM
Mean Squared Error: 2393818753.334541
Root Mean Squared Error: 48926.667098163765
Mean Absolute Error: 36844.42432679947
R2 Score: 0.033571776114130314
Adjusted R2 Score: 0.01797278790760859

Evaluation - Test Set - Polynomial SVM
Mean Squared Error: 8208155139.368153
Root Mean Squared Error: 90598.86941550735
Mean Absolute Error: 38143.54289929179
R2 Score: -2.68016988740199
Adjusted R2 Score: -2.7395710154451622
```

Figure 15: SVR Performance 1

```
Evaluation - Validation Set - RBF SVM
Mean Squared Error: 2533117688.7825375
Root Mean Squared Error: 50330.087311493284
Mean Absolute Error: 37127.663776799076
R2 Score: -0.022665740860232653
Adjusted R2 Score: -0.03917245127763924

Evaluation - Test Set - RBF SVM
Mean Squared Error: 2282214661.9741516
Root Mean Squared Error: 47772.53041209092
Mean Absolute Error: 36235.57698233173
R2 Score: -0.023243047064424482
Adjusted R2 Score: -0.03975907570376225

Evaluation - Validation Set - Sigmoid SVM
Mean Squared Error: 2332573568.583746
Root Mean Squared Error: 48296.72420137567
Mean Absolute Error: 35962.05294357058
R2 Score: 0.05829757251697987
Adjusted R2 Score: 0.043097680070445454

Evaluation - Test Set - Sigmoid SVM
Mean Squared Error: 2101264246.197243
Root Mean Squared Error: 45839.54893099672
Mean Absolute Error: 35047.21742226465
R2 Score: 0.05788703149136554
Adjusted R2 Score: 0.04268051255725702
```

Figure 16: SVR Performance 2

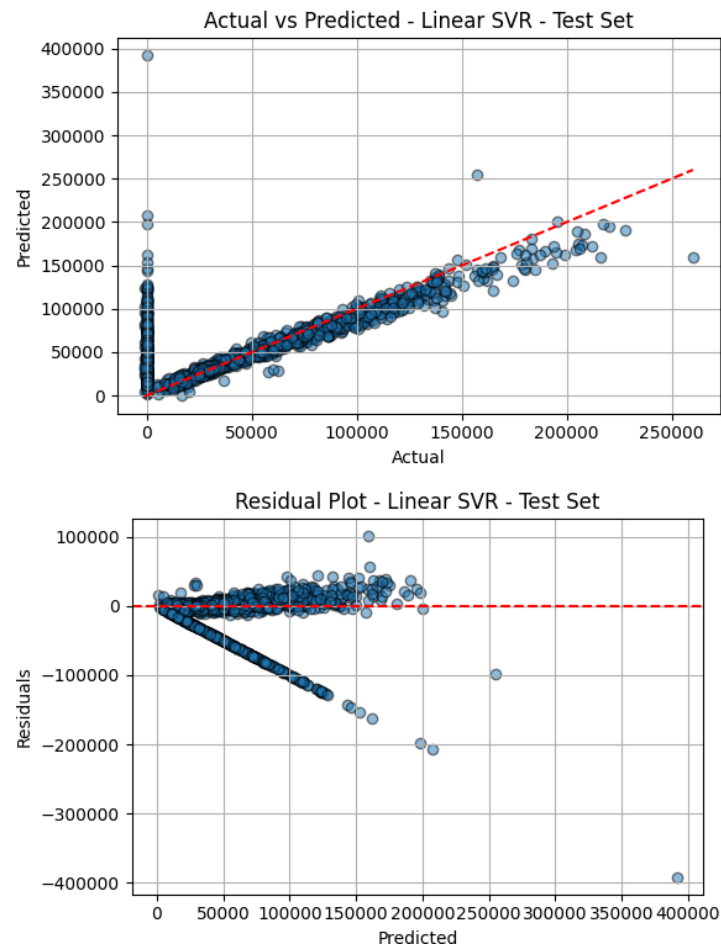


Figure 17: SVR Linear Kernel Visualization

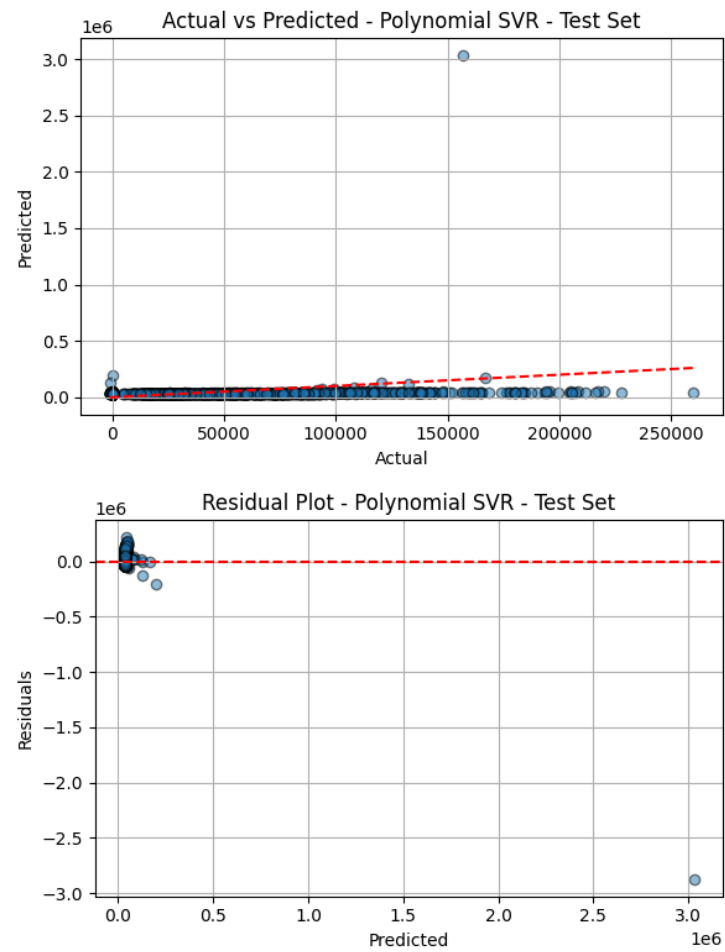


Figure 18: SVR Polynomial Kernel Visualization

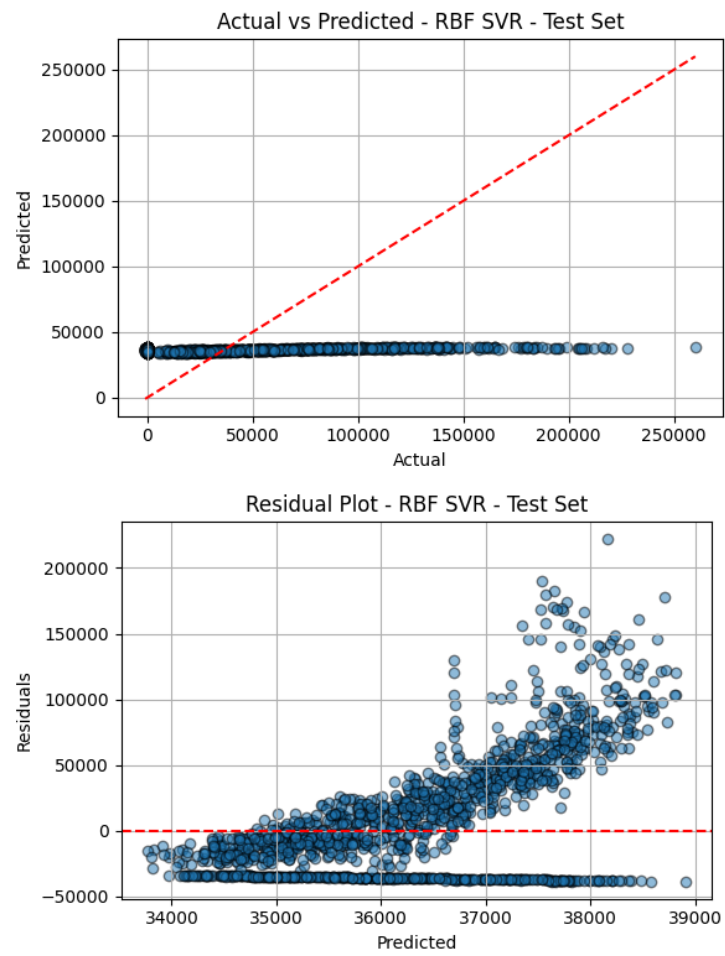


Figure 19: SVR RBF Kernel Visualization

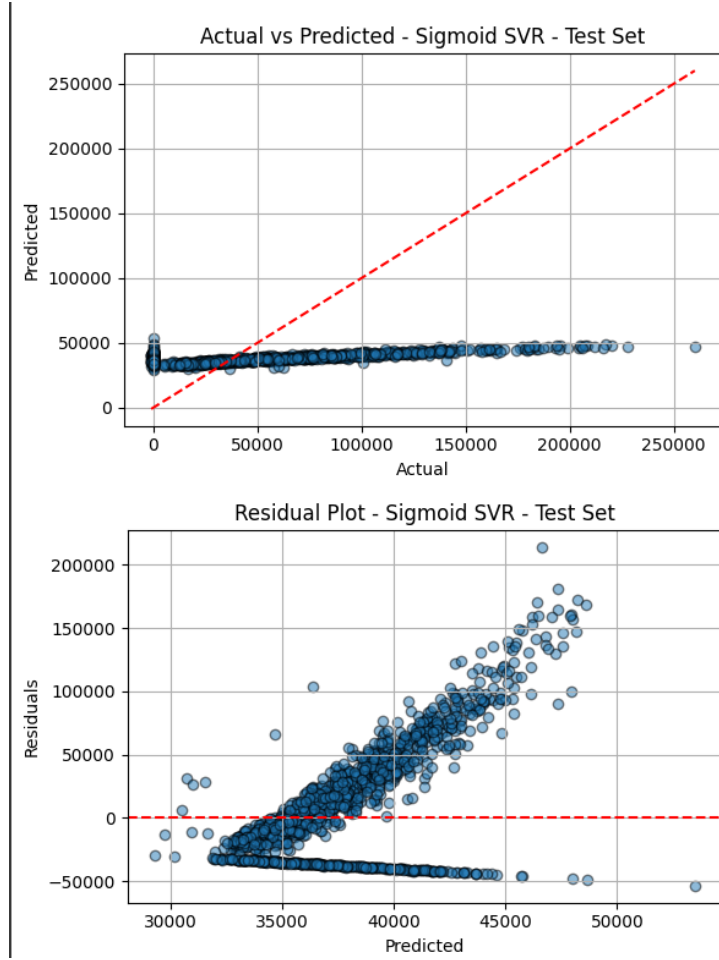


Figure 20: SVR Sigmoid Kernel Visualization

Results Tables

Table 1: Cross-Validation Results ($K = 5$)

Fold	MAE	MSE	RMSE	R^2 Score
Fold 1	21539.37	1.0153e+09	31863.89	0.552684
Fold 2	21627.32	9.6861e+08	31122.58	0.573856
Fold 3	21507.68	1.1811e+09	34366.95	0.488430
Fold 4	21511.18	9.1793e+08	30297.28	0.620162
Fold 5	21709.72	9.9336e+08	31517.46	0.579387
Average	21579.05	1.0153e+09	31833.63	0.562744

Table 2: Summary of Results for Loan Amount Prediction

Description	Student's Result
Dataset Size (after preprocessing)	29660 rows, 23 columns
Train/Test Split Ratio	60% Test, 20% Train, 20% Validation
Feature(s) Used for Prediction	Age, Income, Property, Profession, Type of Employment, Location, Expenses, Credit Score, Defaults, Dependents, DebttoIncomeRatio, LoantoPropertyRatio, FinancialBurden
Model Used	Linear Regression
Cross-Validation Used? (Yes/No)	Yes
If Yes, Number of Folds (K)	5
Reference to CV Results Table	Table 1
Mean Absolute Error (MAE) on Test Set	21714.30955318851
Mean Squared Error (MSE) on Test Set	1017408194.6607875
Root Mean Squared Error (RMSE) on Test Set	31896.836750072685
R^2 Score on Test Set	0.5499689011897946
Adjusted R^2 Score on Test Set	0.5482933750138216
Most Influential Feature(s)	Income, Credit Score, Property Age, FinancialBurden
Observations from Residual Plot	Residuals are mostly near 0, with some randomly scattered in -35000, 10000
Interpretation of Predicted vs Actual Plot	Most points cluster along $y=x$ line with some near it, and some near each axes, indicating a good fit for the most part
Any Overfitting or Underfitting Observed?	No major signs
Brief Justification for above (e.g., training vs test error, residual patterns)	Train and Validation metrics are good and very close to each other, and plots show good fit

2 Support Vector Regression Results

The Support Vector Regression (SVR) models were evaluated using different kernels: Linear, Polynomial, Radial Basis Function (RBF), and Sigmoid. The results indicate that the Linear SVR model achieved a performance comparable to that of the Linear Regression model. In contrast, the Polynomial, RBF, and Sigmoid kernel SVR models yielded noticeably lower R^2 scores, suggesting that the relationship between the features and the target variable is predominantly linear in nature.

Best Practices

- Normalize or standardize features when necessary.
- Split data into train, validation, and test sets.
- Use K-Fold Cross Validation to get more reliable performance estimates.
- Visualize residuals and feature importance to interpret model behavior.

Learning Outcomes

- Understood and applied linear regression for regression tasks.
- Evaluated model performance using MAE, MSE, RMSE, R^2 .
- Gained experience with cross-validation and model interpretation.
- Learned about regularization and comparison with Ridge, Lasso, ElasticNet.
- Learned to apply SVR and its variations, while comparing it with Linear Regression.